



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Colin Oakes
17th May 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- SpaceX launch data was collected via the SpaceX REST API and Wikipedia web scraping, then cleaned and wrangled for analysis
- EDA, interactive maps, a Plotly Dash dashboard, and classification models (Logistic Regression, SVM, Decision Tree, KNN) were used to predict first-stage landing success

Introduction

- SpaceX advertises Falcon 9 launches at \$62M vs competitors at \$165M+ — cost savings depend on successfully recovering the first stage booster
- Can we use publicly available data to predict whether the first stage will land successfully, and determine which factors most influence launch success?

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
 - Describe how data was collected
- Perform data wrangling
 - Describe how data was processed
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - How to build, tune, evaluate classification models

Data Collection

- Data was collected from two sources: the SpaceX REST API (launch records, rocket specs, payload data) and Wikipedia (historical Falcon 9 launch table via web scraping)
- <https://github.com/coathangers/Python/blob/main/jupyter-labs-spacex-data-collection-api-v2.ipynb>

Data Collection – SpaceX API

- Used the SpaceX REST API to request launch data including flight number, date, payload mass, orbit, launch site, and landing outcome. Responses parsed from JSON into a Pandas DataFrame.
- **GitHub URL:**
<https://github.com/coathangers/Python/blob/main/jupyter-labs-spacex-data-collection-api-v2.ipynb>

Send GET Request to SpaceX API



Receive JSON Response



Normalize JSON → Pandas DataFrame

Data Collection - Scraping

- Used BeautifulSoup to scrape the Falcon 9 launch history table from Wikipedia. Extracted columns including launch date, payload, launch site, and landing outcome, then stored results in a DataFrame.

- **GitHub URL:**
<https://github.com/coathangers/Python/blob/main/jupyter-labs-webscraping-v2.ipynb>

Scrape Wikipedia Table (BeautifulSoup)



Clean & Merge Both Sources



Export to spacex_launch_dash.csv

Data Wrangling

- Handled missing values (e.g. filled missing Payload Mass with column mean). Created a binary class column (1 = successful landing, 0 = failure) from the landing outcome field. Filtered out irrelevant booster versions.
- Applied one-hot encoding to categorical features (orbit type, launch site, grid fins, etc.) to prepare data for machine learning models.
- **GitHub URL:** <https://github.com/coathangers/Python/blob/main/labs-jupyter-spacex-Data%20wrangling-v2.ipynb>

EDA with Data Visualization

- Plotted scatter plots (Flight Number vs Launch Site, Payload vs Launch Site, Flight Number vs Orbit Type), bar charts (success rate by orbit), and a line chart of yearly average success rate to identify trends and relationships in the data.
- **GitHub URL:** <https://github.com/coathangers/Python/blob/main/jupyter-labs-eda-dataviz-v2.ipynb>

EDA with SQL

- Queried the SpaceX dataset using SQL:
 - Distinct launch sites
 - Total payload by NASA boosters
 - Average payload for F9 v1.1
 - First successful drone ship landing date
 - Boosters with max payload
 - Landing outcome rankings between 2010-2017.
- **GitHub URL:** <https://github.com/coathangers/Python/blob/main/jupyter-labs-eda-sql-edx-sqlite-v2.ipynb>

Build an Interactive Map with Folium

- Added circle markers for all four launch sites on a global Folium map, colour-coded markers by launch outcome (green = success, red = failure), and calculated distances from each site to nearby railways, highways, and coastlines using MousePosition and Marker Cluster plugins.
- Proximity markers were added to identify geographic patterns — all sites are near coastlines and transport links, which is important for safety and logistics.
- **GitHub URL:** <https://github.com/coathangers/Python/blob/main/lab-jupyter-launch-site-location-v2.ipynb>

Build a Dashboard with Plotly Dash

- Built an interactive dashboard with a launch site dropdown and payload range slider. Added a pie chart showing success counts by site and a scatter plot of payload vs outcome coloured by booster version.
- The dropdown and slider allow filtering by site and payload to interactively explore which conditions lead to successful landings across different booster versions.
- **GitHub URL:**
https://github.com/coathangers/Python/blob/main/spacex_dashboard.ipynb

Predictive Analysis (Classification)

- Trained and evaluated four classification models: Logistic Regression, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbours (KNN). Used GridSearchCV for hyperparameter tuning and evaluated with accuracy score and confusion matrix.
- Data was split 80/20 for training and testing. Features were standardised using StandardScaler. The best model was selected based on test set accuracy.
- **GitHub URL:** <https://github.com/coathangers/Python/blob/main/SpaceX-Machine-Learning-Prediction-Part-5-v1.ipynb>

Results

Exploratory data analysis results

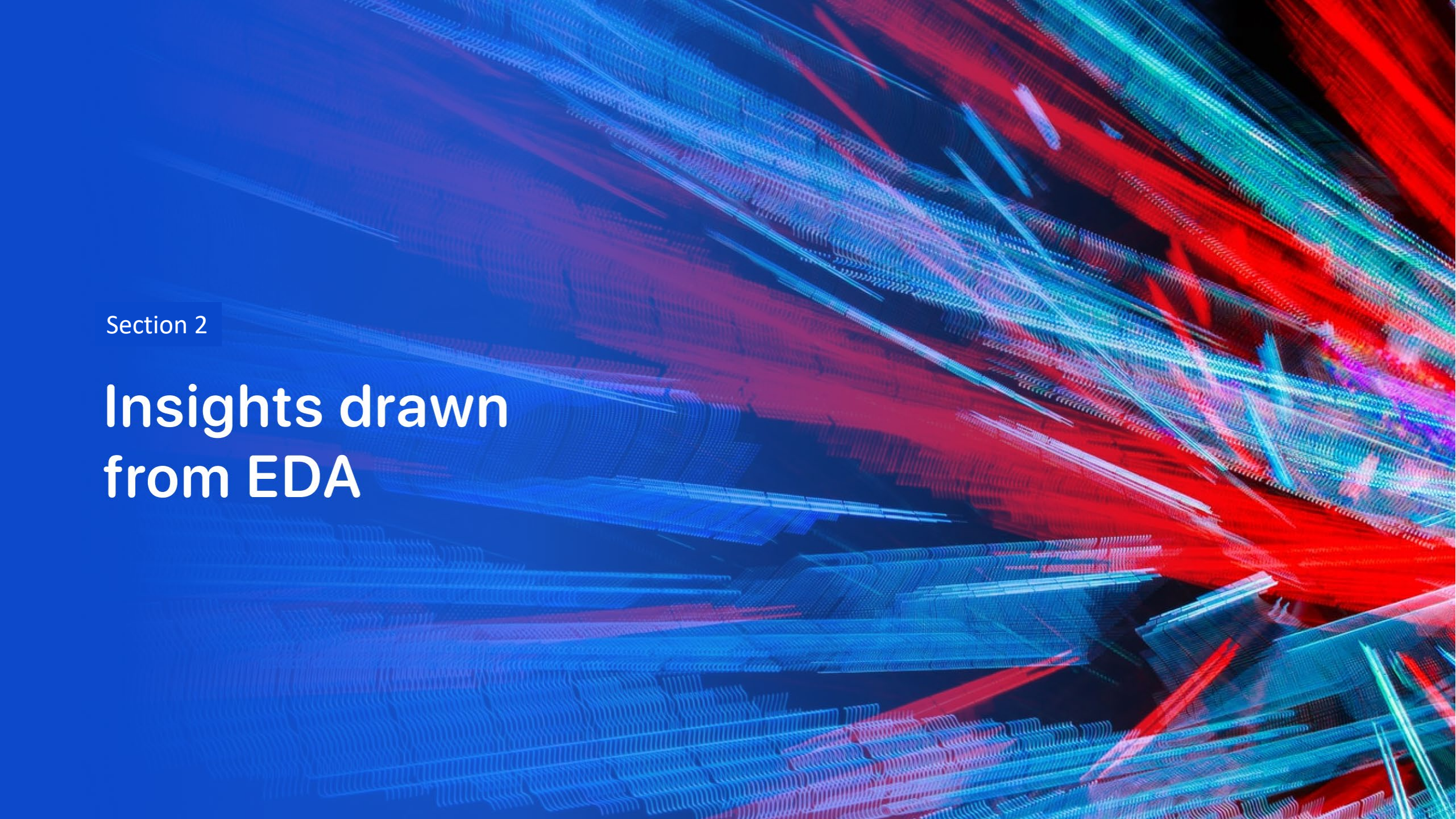
Used Matplotlib and Seaborn to visualise relationships between launch variables such as flight number, payload mass, orbit type and launch site

Interactive analytics demo in screenshots

Built an interactive Folium map and a Plotly Dash dashboard to explore launch outcomes geographically and filter by site and payload range

Predictive analysis results

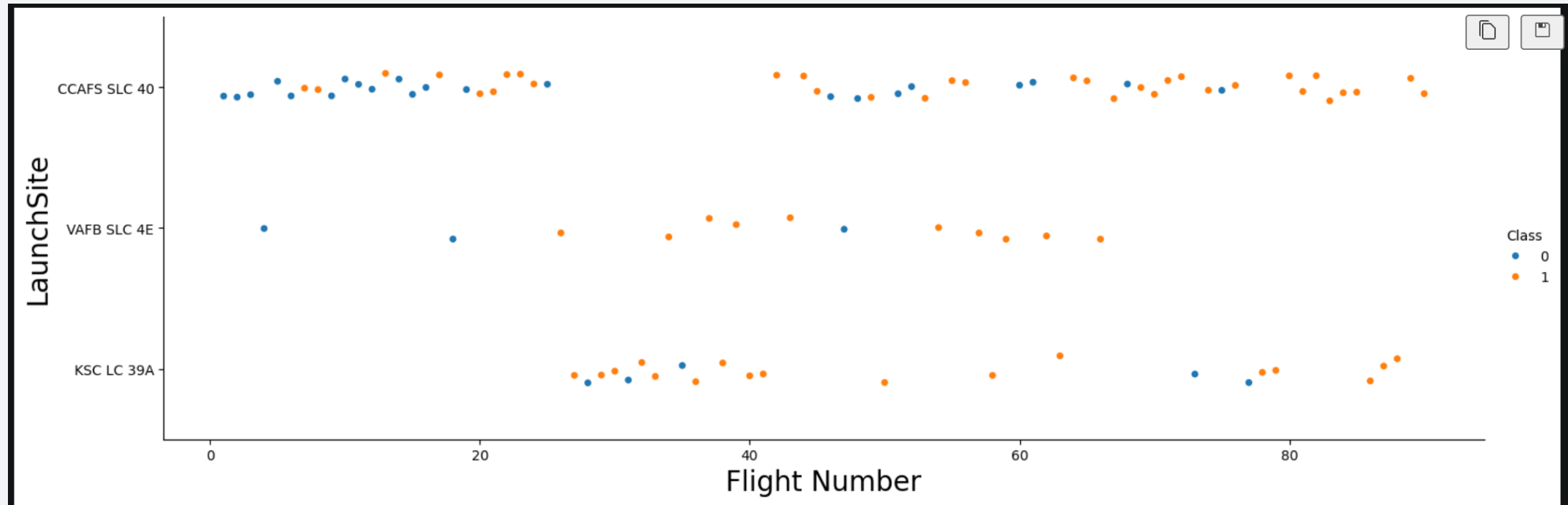
Trained and compared four classification models (Logistic Regression, SVM, Decision Tree, KNN) using GridSearchCV to find the best hyperparameters, evaluated using accuracy score and confusion matrix

The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of red and cyan. A faint, light blue grid pattern is also visible, particularly in the lower half of the image. The overall effect is dynamic and technological.

Section 2

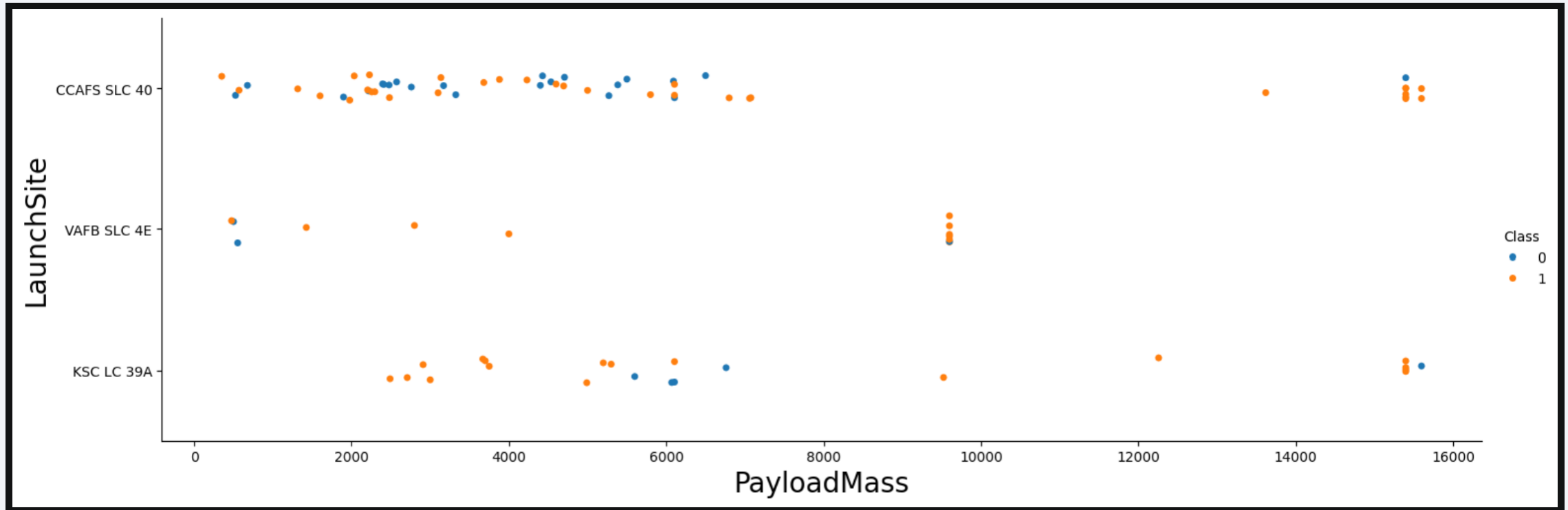
Insights drawn from EDA

Flight Number vs. Launch Site



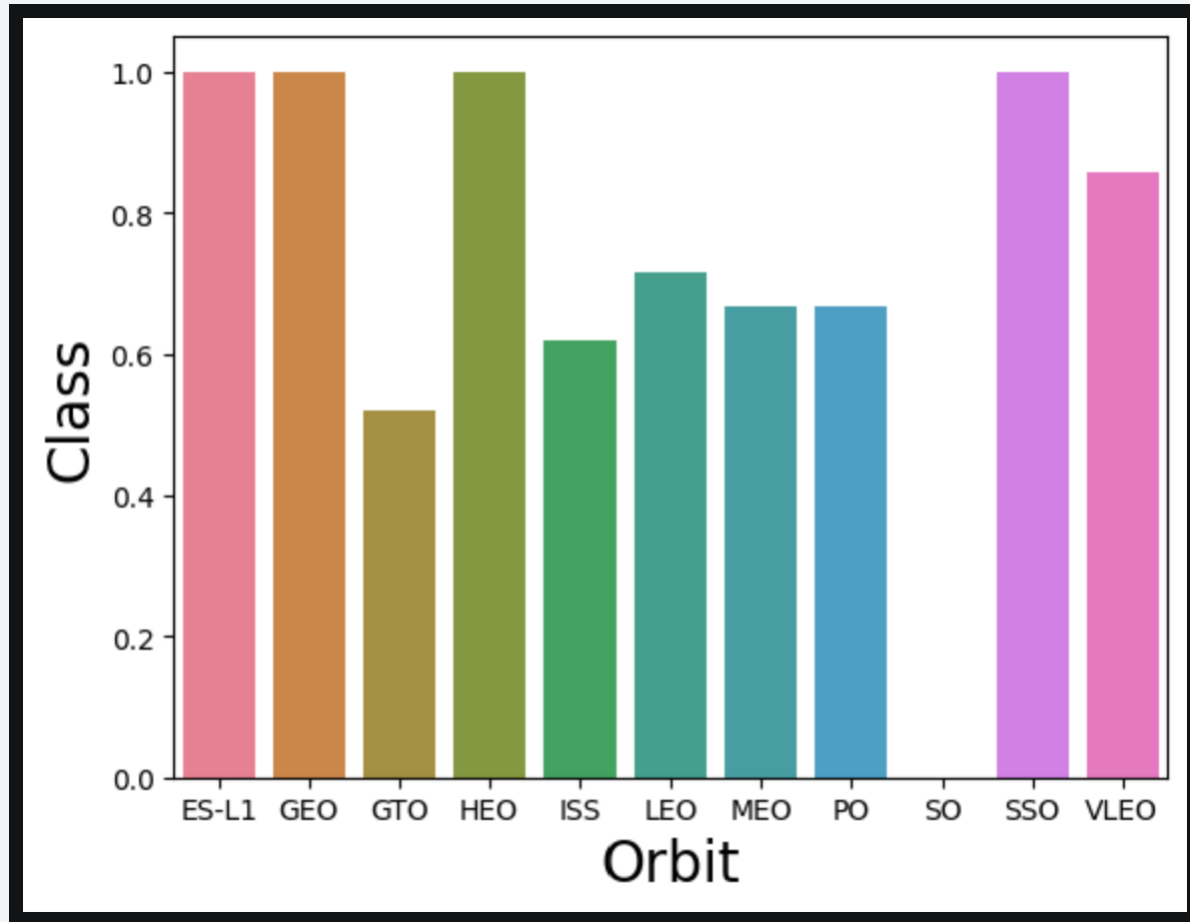
There is a clear positive trend in launch success as flight number increases, suggesting that SpaceX's reliability improved significantly with experience.

Payload vs. Launch Site



If you observe Payload Vs. Launch Site scatter point chart you will find for the VAFB-SLC launchsite there are no rockets launched for heavy payload mass (greater than 10000).

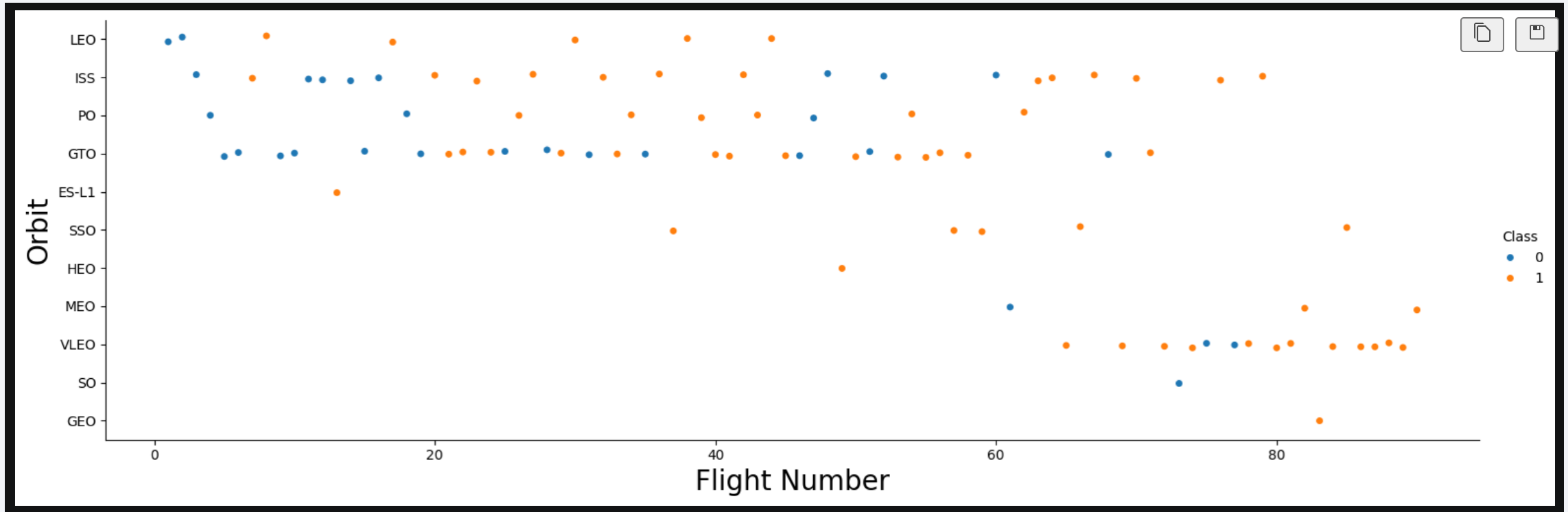
Success Rate vs. Orbit Type



Orbits with 100% success rates:

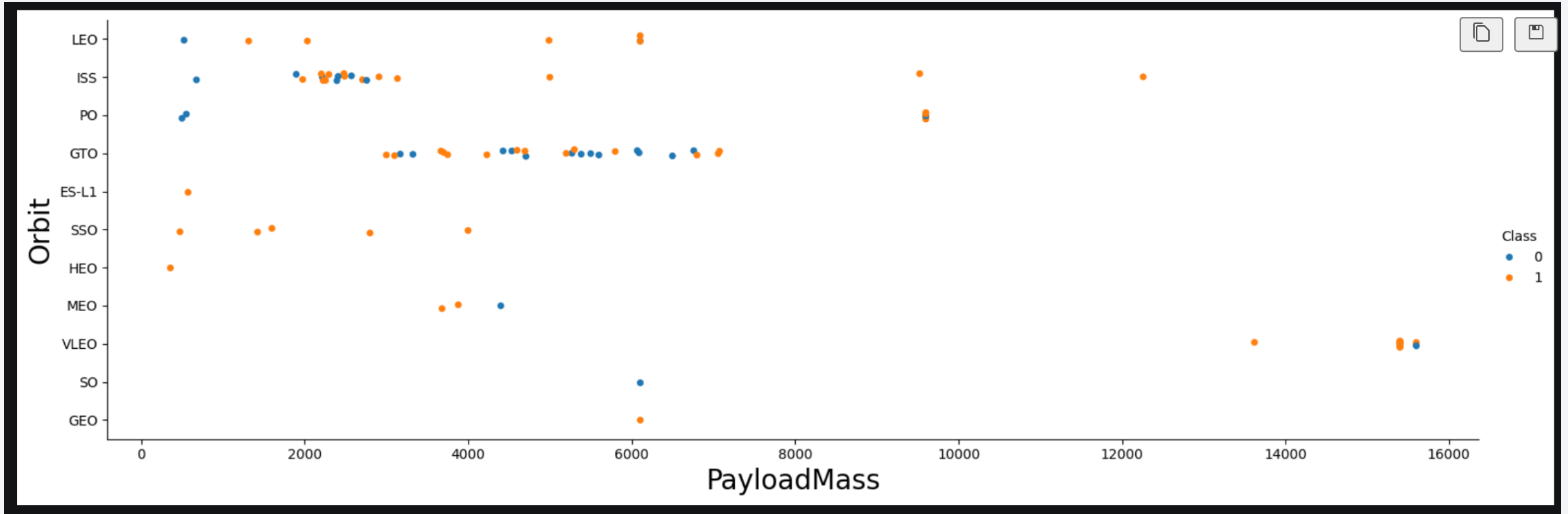
- ES-L1
- GEO
- HEO
- SSO

Flight Number vs. Orbit Type



In the LEO orbit the Success appears related to the number of flights; on the other hand, there seems to be no relationship between flight number when in GTO orbit.

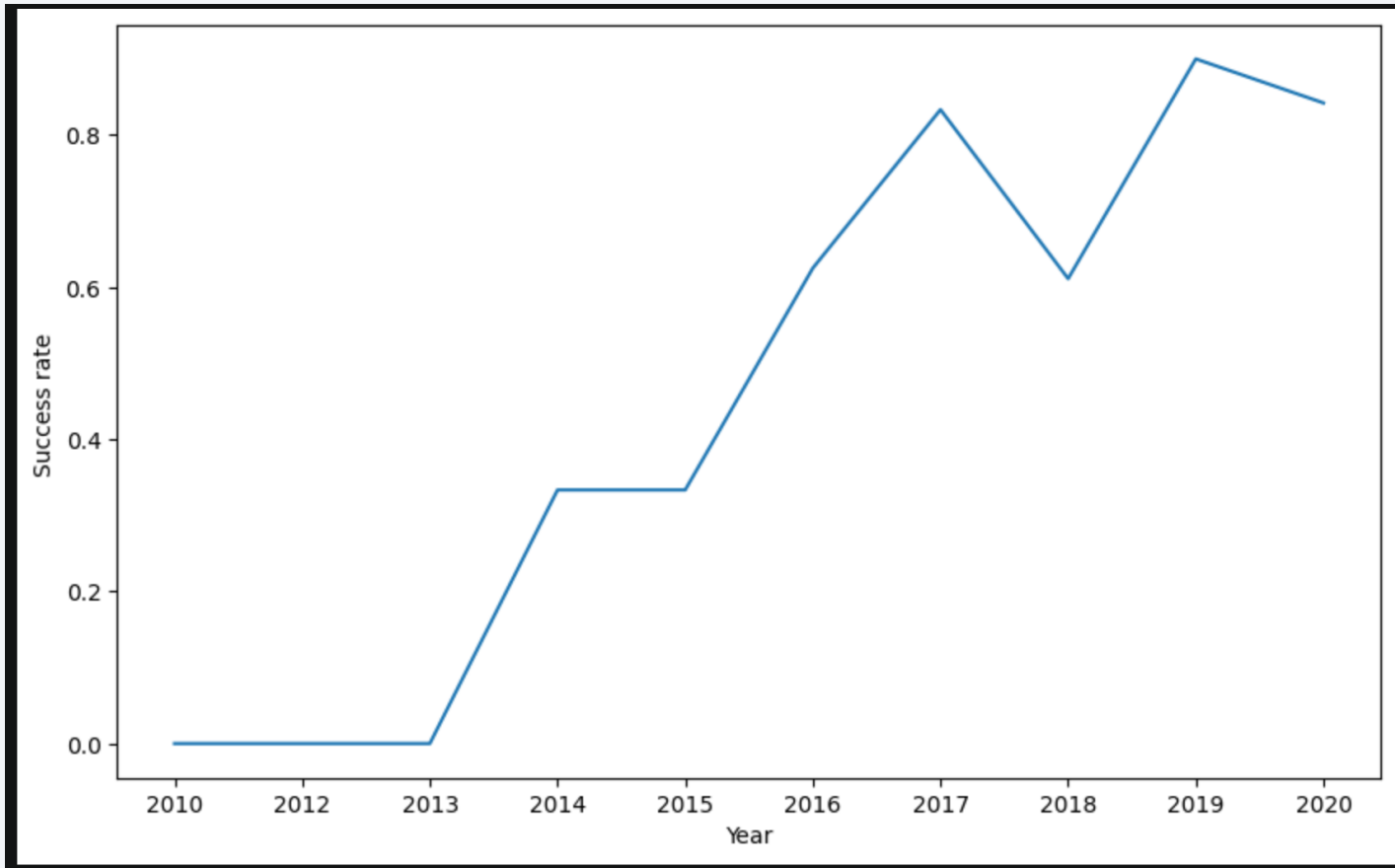
Payload vs. Orbit Type



With heavy payloads the successful landing or positive landing rate are more for Polar, LEO and ISS.

However for GTO we cannot distinguish this well as both positive landing rate and negative landing(unsuccesful mission) are both there here.

Launch Success Yearly Trend



You can observe that the success rate since 2013 kept increasing till 2017 (stable in 2014) and after 2015 it started increasing.

All Launch Site Names

Launch Sites

- CCAFS LC-40
- VAFB SLC-4E
- KSC LC-39A
- CCAFS SLC-40

Four unique launch sites were identified in the dataset.

```
pd.read_sql("SELECT DISTINCT Launch_Site FROM SPACEXTABLE",con)
```

Launch Site Names Begin with 'KSC'

Date	Time (UTC)	Booster_Versior	Launch_Site	Payload	Payload Mass KG	Orbit	Customer	Mission_Outcome	Landing_Outcome
19/02/2017	14:39:00	F9 FT B1031.1	KSC LC-39A	SpaceX CRS-10	2490	LEO (ISS)	NASA (CRS)	Success	Success (ground pad)
16/03/2017	06:00:00	F9 FT B1030	KSC LC-39A	EchoStar 23	5600	GTO	EchoStar	Success	No attempt
30/03/2017	22:27:00	F9 FT B1021.2	KSC LC-39A	SES-10	5300	GTO	SES	Success	Success (drone ship)
01/05/2017	11:15:00	F9 FT B1032.1	KSC LC-39A	NROL-76	5300	LEO	NRO	Success	Success (ground pad)
15/05/2017	23:21:00	F9 FT B1034	KSC LC-39A	Inmarsat-5 F4	6070	GTO	Inmarsat	Success	No attempt

All 3 attempted landing outcomes were successful in this sample of launches from KSC.

```
pd.read_sql("SELECT * FROM SPACEXTABLE WHERE Launch_Site LIKE  
'KSC%' LIMIT 5",con)
```

Total Payload Mass

Total Payload Mass carried boosters launched by NASA = 45596KG

NASA missions include ISS resupply and scientific payloads.

```
pd.read_sql("SELECT SUM(PAYLOAD_MASS__KG_) AS 'Total  
Payload Mass' FROM SPACEXTABLE WHERE Customer = 'NASA  
(CRS)'",con)
```

Average Payload Mass by F9 v1.1

Average Payload Mass by booster F9 v1.1 = 2534.67KG

The v1.1 booster handled moderate payloads before the more capable Full Thrust variant was introduced.

```
pd.read_sql("SELECT AVG(PAYLOAD_MASS__KG_) AS 'Average  
Payload Mass by booster F9 v1.1' FROM SPACE_TABLE WHERE  
Booster_Version LIKE '%F9 v1.1%',con)
```

First Successful Ground Landing Date

First Drone ship success = 8th April 2016

```
pd.read_sql("SELECT MIN(Date) AS 'Drone ship success date'  
FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (drone  
ship)'",con)
```

Total Drone ship landing successes = 14

```
pd.read_sql("SELECT COUNT(*) AS 'Drone ship successes' FROM  
SPACEXTABLE WHERE Landing_Outcome = 'Success (drone  
ship)'",con)
```

Successful Drone Ship Landing with Payload between 4000 and 6000

Booster Version
F9 FT B1032.1
F9 B4 B1040.1
F9 B4 B1043.1

These boosters successfully landed on drone ships while carrying payloads between 4000 and 6000 kg, demonstrating the capability to recover heavier-mission boosters.

```
pd.read_sql("SELECT Booster_Version FROM SPACEXTABLE  
WHERE Landing_Outcome = 'Success (ground pad)' AND  
PAYLOAD_MASS__KG_ > 4000 AND PAYLOAD_MASS__KG_ <  
6000",con)
```


Total Number of Successful and Failure Mission Outcomes

Mission Outcome	Count
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Success rate has improved significantly in recent years

```
pd.read_sql("SELECT Mission_Outcome, COUNT(*) AS Count  
FROM SPACEXTABLE GROUP BY Mission_Outcome;",con)
```

Boosters Carried Maximum Payload

Booster Version

- F9 B5 B1048.4
- F9 B5 B1049.4
- F9 B5 B1051.3
- F9 B5 B1056.4
- F9 B5 B1048.5
- F9 B5 B1051.4
- F9 B5 B1049.5
- F9 B5 B1060.2
- F9 B5 B1058.3
- F9 B5 B1051.6
- F9 B5 B1060.3
- F9 B5 B1049.7

These were typically GTO or heavy-lift missions

```
pd.read_sql("SELECT Booster_Version  
FROM SPACEXTABLE WHERE  
PAYLOAD_MASS__KG_ = (SELECT  
MAX(PAYLOAD_MASS__KG_) FROM  
SPACEXTABLE);",con)
```

(Subquery was used)

2017 Launch Records

Month	Landing_Outcome	Booster_Version	Launch_Site
February	Success (ground pad)	F9 FT B1031.1	KSC LC-39A
May	Success (ground pad)	F9 FT B1032.1	KSC LC-39A
June	Success (ground pad)	F9 FT B1035.1	KSC LC-39A
August	Success (ground pad)	F9 B4 B1039.1	KSC LC-39A
September	Success (ground pad)	F9 B4 B1040.1	KSC LC-39A
December	Success (ground pad)	F9 FT B1035.2	CCAFS SLC-40

Note: SQLite does not support monthnames. So you need to use substr(Date,6,2) for month, substr(Date,9,2) for date, substr(Date,0,5),='2017' for year.

- SELECT
- CASE substr(Date, 6, 2)
- WHEN '01' THEN 'January'
- WHEN '02' THEN 'February'
- WHEN '03' THEN 'March'
- WHEN '04' THEN 'April'
- WHEN '05' THEN 'May'
- WHEN '06' THEN 'June'
- WHEN '07' THEN 'July'
- WHEN '08' THEN 'August'
- WHEN '09' THEN 'September'
- WHEN '10' THEN 'October'
- WHEN '11' THEN 'November'
- WHEN '12' THEN 'December'
- END AS Month,
- Landing_Outcome,
- Booster_Version,
- Launch_Site
- FROM SPACEXTBL
- WHERE Landing_Outcome = 'Success (ground pad)'
- AND substr(Date, 1, 4) = '2017';

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

No attempt was the most frequent outcome in the early period, followed by drone ship failures. Successful ground pad and drone ship landings increased towards 2017. [Add your query result screenshot here]

Landing_Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

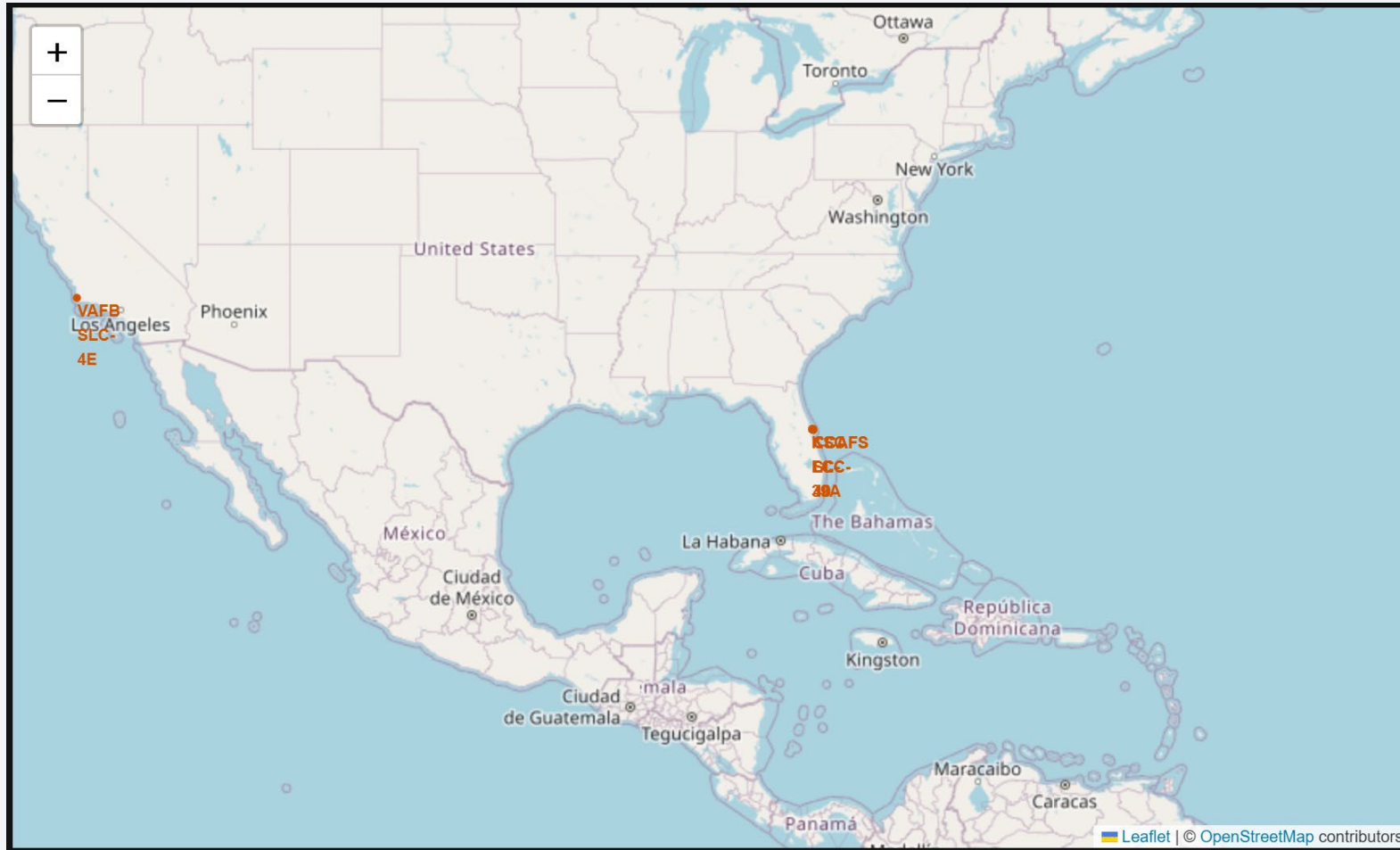
```
pd.read_sql("SELECT Landing_Outcome,
COUNT(*) AS Count FROM
SPACEXTABLE WHERE Date BETWEEN
'2010-06-04' AND '2017-03-20' GROUP
BY Landing_Outcome ORDER BY Count
DESC;",con)
```

Section 3

Launch Sites Proximities Analysis

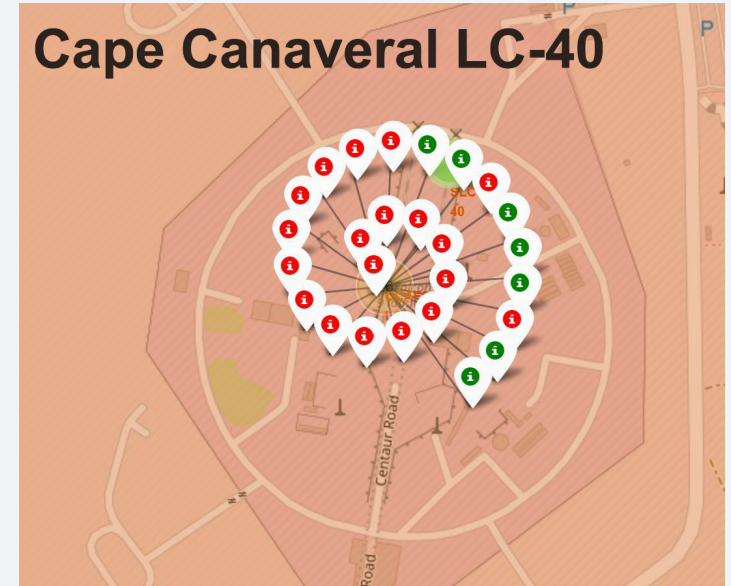
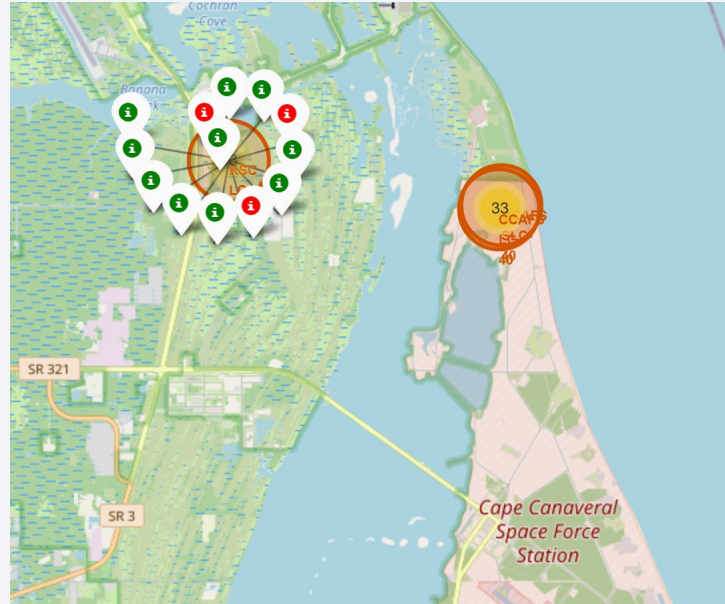
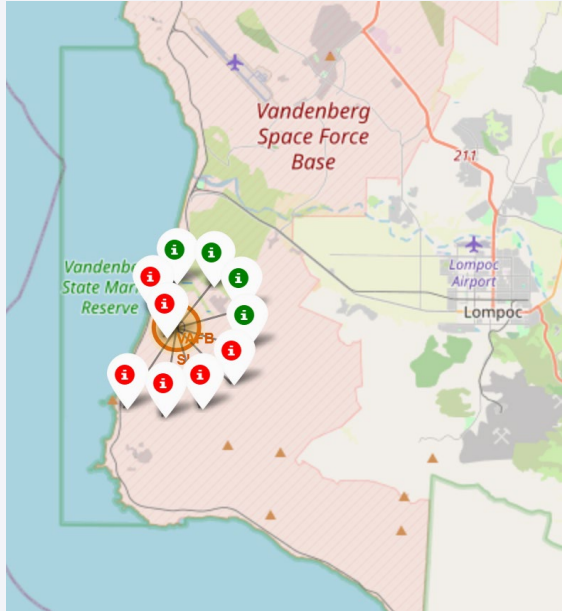


Map of Launch Site Locations



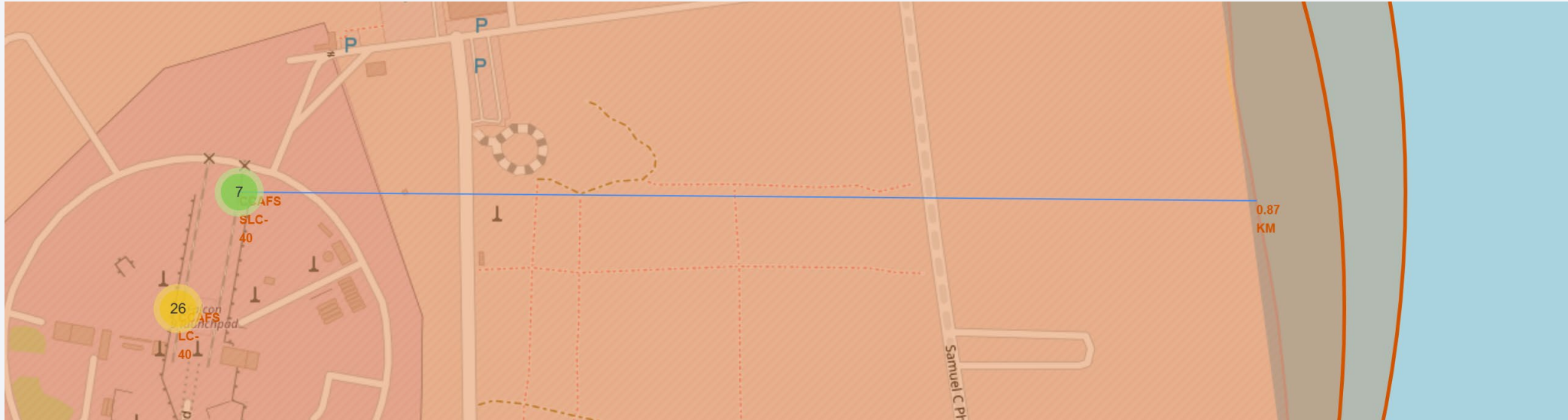
- Launch Sites in both Florida and California are in close proximity to the coast-lines
- Florida Launch Sites are closer to the equator, California Launch Sites are slightly further north

Successful/failed Launches

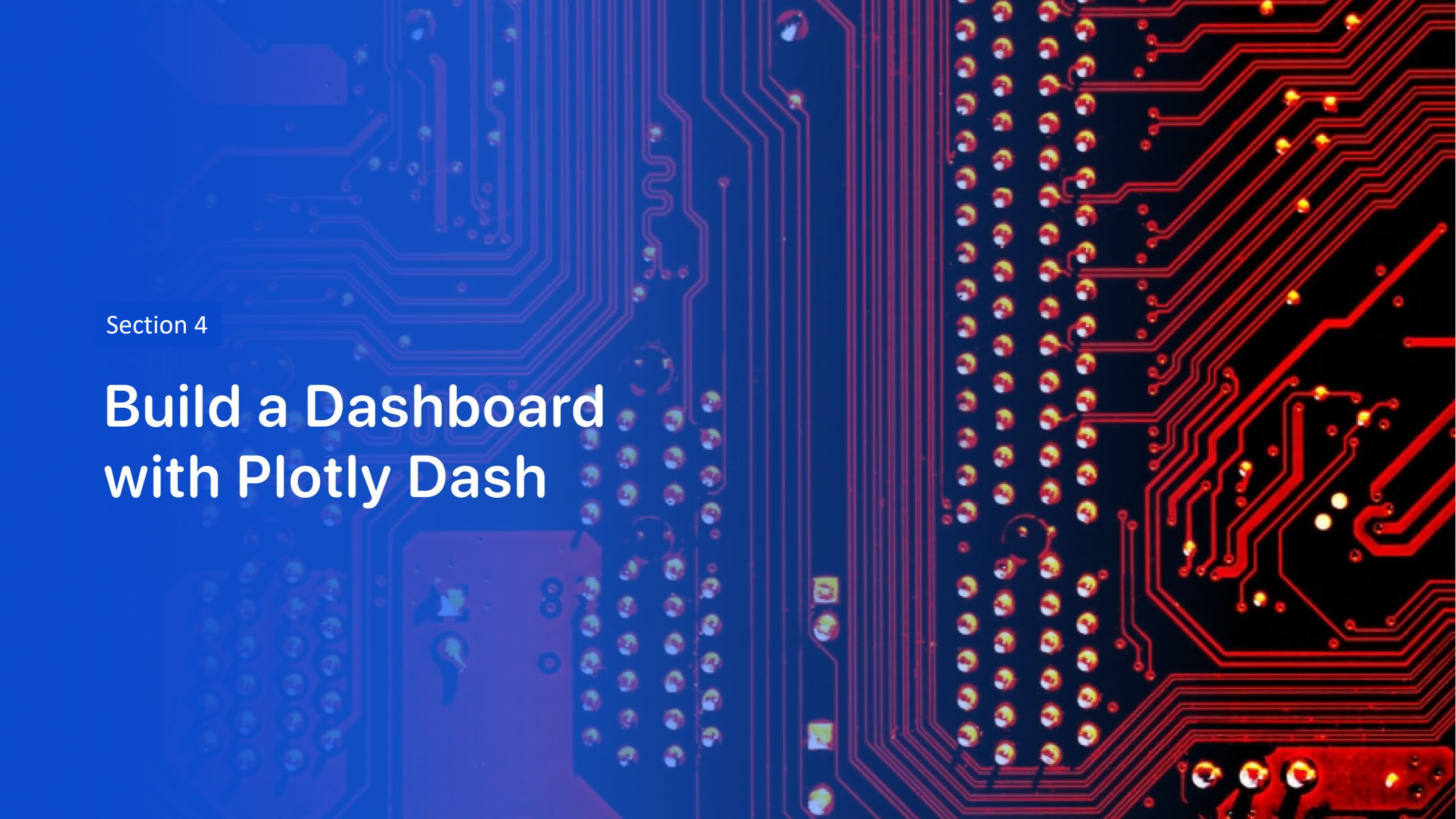


Cape Canaveral Space Force Station LC-40 has high failure rate whereas LC-39A has high success rate

CCAFS SLC-40 Proximity to Coastline



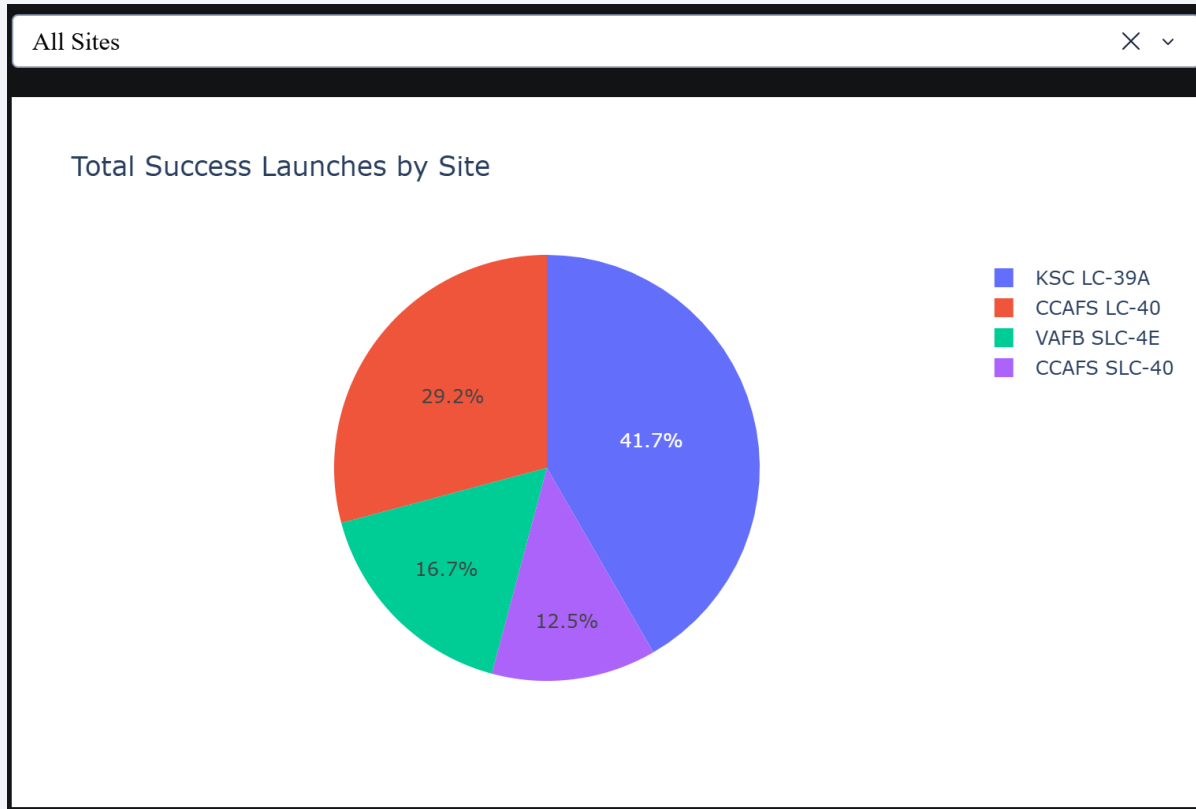
All sites are very close to the coastline, SLC-40 being less than 1km away. This is essential for over-ocean launches and booster recovery at sea.



Section 4

Build a Dashboard with Plotly Dash

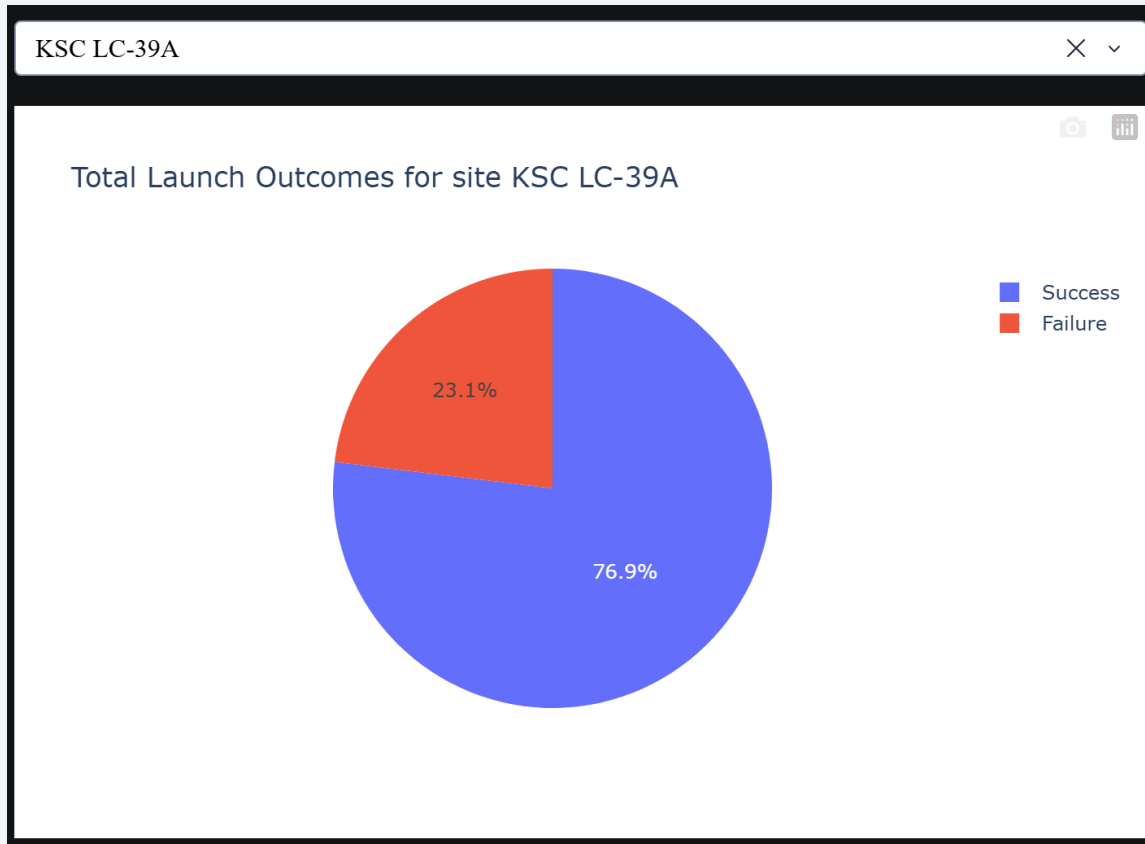
Total Launch Site Successes



Q. Which site has the **largest** successful launches?

A. KSC LC-39A

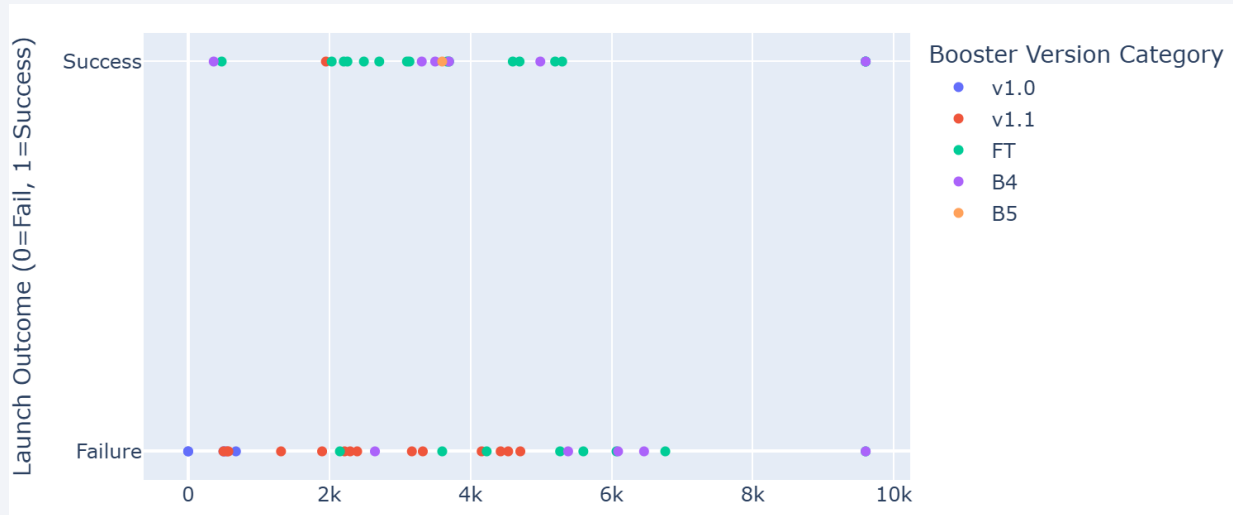
Highest Launch Site Success Rate



Q: Which site has the **highest success rate**?

A: KSC LC-39A

Payload vs. Launch Outcome

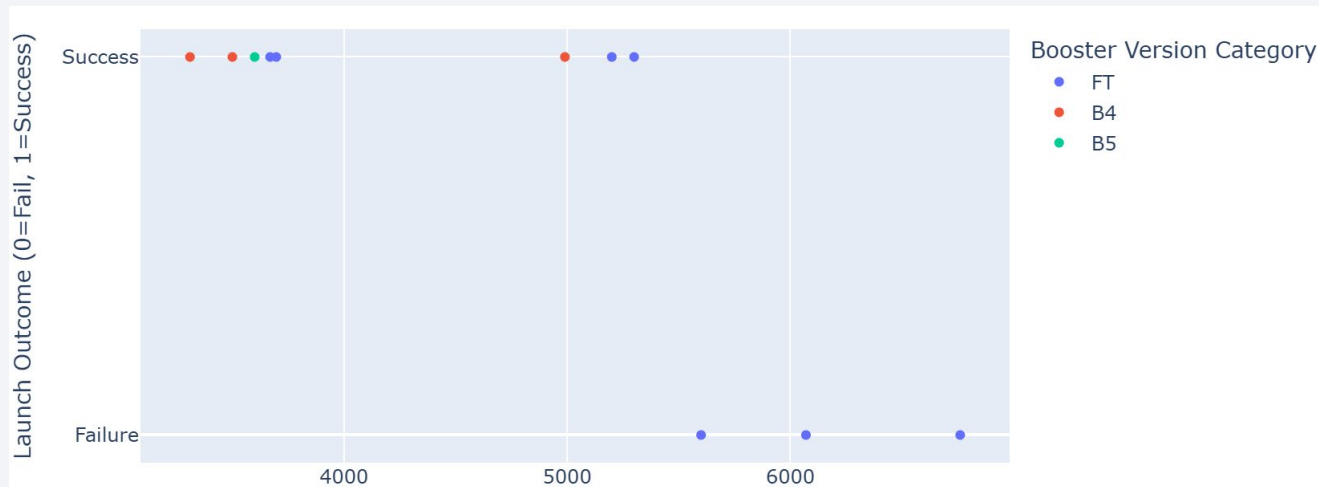


Q: Which payload range has the **highest** success rate?

A: 2k-4k

Q: Which payload range has the **lowest** success rate?

A: 6k-8k



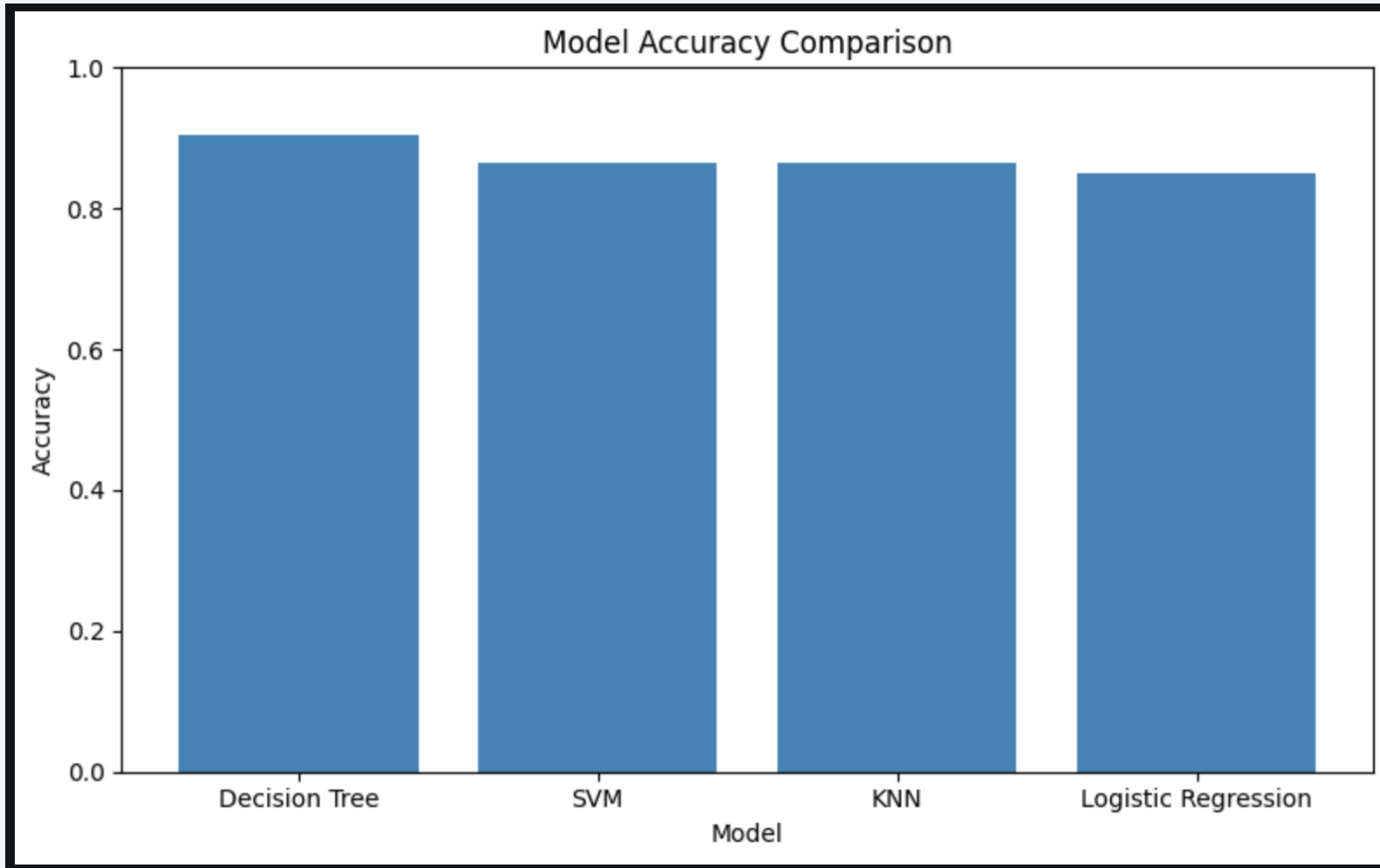
Q: Which F9 Booster version has the highest success rate?

A: FT

Section 5

Predictive Analysis (Classification)

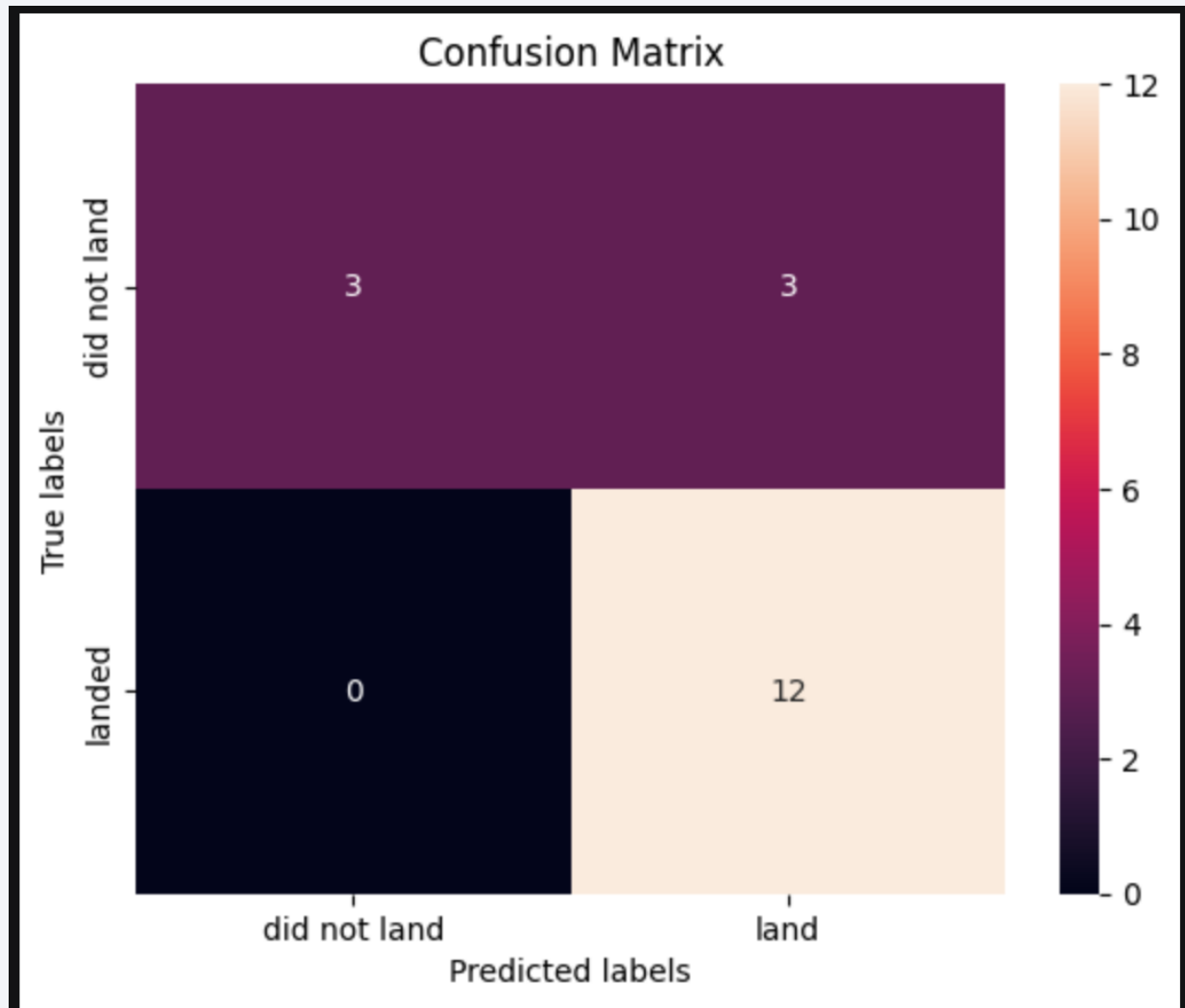
Classification Accuracy



Decision Tree has the highest accuracy therefore yields the best results

0.91

Confusion Matrix



Parameter	Value
criterion	entropy
max_depth	4
max_features	sqrt
min_samples_leaf	1
min_samples_split	10
splitter	best

Conclusions

- Decision Tree achieved the highest test accuracy of 93%, likely due to the dataset's clear threshold-based features such as payload mass and orbit type, combined with regularisation through `max_depth` and `min_samples_split` to prevent overfitting.
- SpaceX launch success depends on fairly distinct conditions — payload mass, orbit type, launch site. Decision trees are good at splitting on these kinds of categorical/threshold rules naturally.

Appendix

Predictive analysis results:

Model	Score	Accuracy ^v
Decision Tree	0.833333	0.930357
SVM	0.833333	0.864286
KNN	0.777778	0.864286
Logistic Regression	0.833333	0.850000

Python Bar Chart Snippet:

```
scores = {
    'Model': ['KNN', 'SVM', 'Decision
Tree', 'Logistic Regression'],
    'Score': [
        knn_cv.score(X_test, Y_test),
        svm_cv.score(X_test, Y_test),
        tree_cv.score(X_test, Y_test),
        logreg_cv.score(X_test, Y_test)
    ],
    'Accuracy': [
        knn_cv.best_score_,
        svm_cv.best_score_,
        tree_cv.best_score_,
        logreg_cv.best_score_
    ]
}

results_df = pd.DataFrame(scores)
results_df =
results_df.sort_values(by='Accuracy',
ascending=False)
results_df
```

Thank you!

